

Ain Shams University

Faculty of Engineering - CSE354 Distributed Computing

Distributed LLM Inference System

Efficient Load Balancing and GPU Cluster Task Distribution for Handling
1000+ Concurrent LLM Requests

Project Type	Distributed Computing Course Project
Main Repository	https://github.com/karimmarwanw/DistributedLLM
Implementation	Python, FastAPI, Ollama, Qdrant, Bash scripts
Prepared For	CSE354 - 2nd Semester 2025/2026
Prepared By	DistributedLLM Project Team

This report explains the problem, architecture, implementation, testing, fault tolerance, limitations, and future work for the DistributedLLM project.

Table of Contents

1. Executive Summary
2. Project Requirements Mapping
3. System Architecture
4. Component Design
5. Load Balancing and GPU Task Distribution
6. LLM Inference and RAG Integration
7. Fault Tolerance
8. Testing and Evaluation
9. Limitations and Future Work
10. Conclusion

1. Executive Summary

DistributedLLM is a distributed LLM serving prototype designed to handle high-concurrency AI requests. The system simulates a real cluster where a client layer submits many requests, a load balancer selects healthy master nodes, master schedulers assign tasks to worker nodes, workers retrieve optional RAG context, and the LLM layer produces a response using either Ollama or a fast simulation path.

The design focuses on the exact goals from the project description: efficient load balancing, GPU cluster task distribution, LLM inference, RAG support, scalability, performance monitoring, and fault tolerance. The implementation is hosted on GitHub at <https://github.com/karimmarwanw/DistributedLLM>.

2. Project Requirements Mapping

Required Point	How the Project Handles It
Load balancing	The load balancer supports Round Robin, Least Connections, and Load-Aware strategies.
GPU cluster task distribution	Two master schedulers distribute requests across four worker services representing GPU nodes.
LLM inference handling	Workers call Ollama for real local inference or simulation mode for high-throughput experiments.
RAG integration	Documents are ingested into Qdrant and retrieved as context before inference.
Scalability	The threaded load generator can simulate 100 to 1000+ concurrent requests in simulation mode.
Fault tolerance	Worker failures are retried by masters; master failures are retried by the load balancer.
Monitoring	Client output reports latency, throughput, master/worker distribution, model distribution, and GPU utilization.
Testing and evaluation	Scripts cover load tests, strategy comparison, RAG retrieval, worker failover, and master failover.
Documentation	README, project report, code review notes, and this PDF describe the system and its tradeoffs.

User Story Coverage

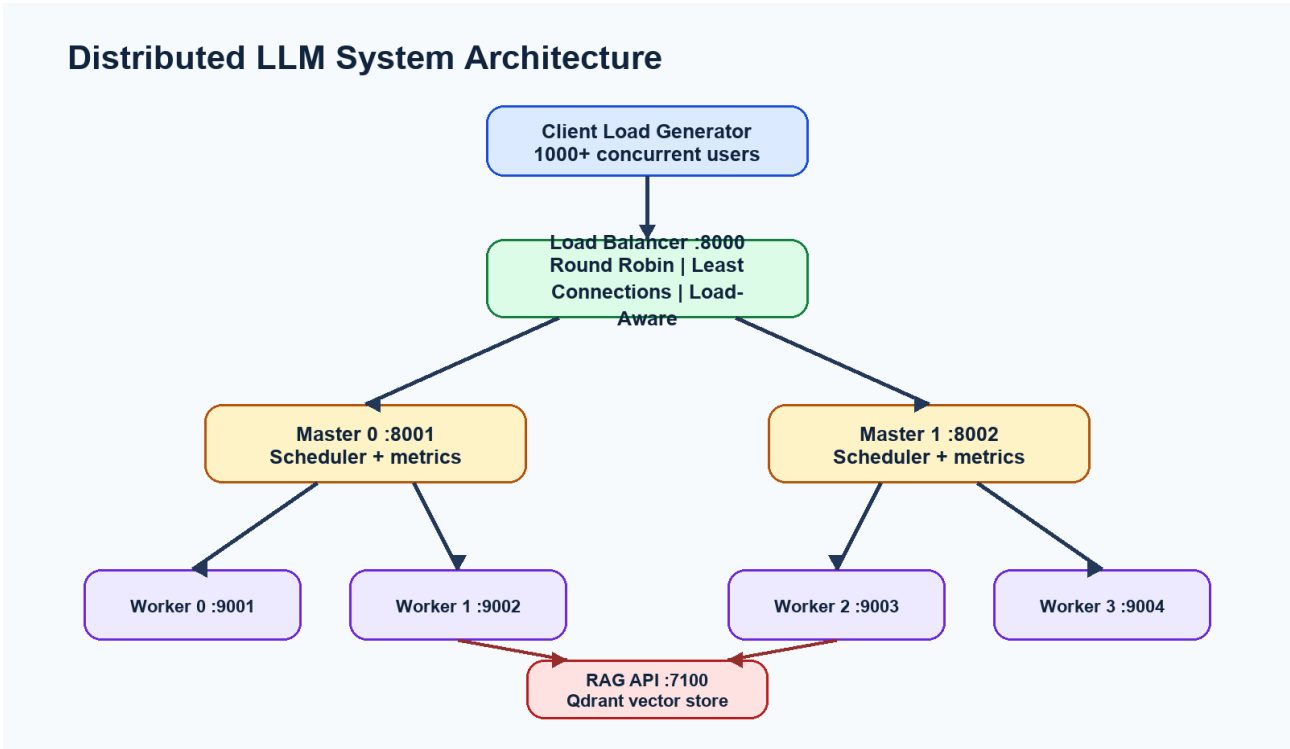
Project User Story	Implemented Support
Send multiple AI requests simultaneously and receive fast responses.	The client creates concurrent request threads, and simulation mode supports high-volume testing.
Remain responsive under heavy load.	Load balancing spreads traffic across masters and worker pools; metrics show throughput and latency.
Continue functioning if some nodes fail.	Worker and master failover scripts verify retry paths under partial failure.
Return consistent and accurate LLM responses.	Ollama mode uses local LLM inference, while RAG injects document context when relevant.
Monitor performance and resource utilization.	Health endpoints, logs, client metrics, and GPU utilization summaries expose system status.

Implementation Phase Coverage

Phase from Description	Project Artifact
Architecture design	Service topology, local/distributed modes, and generated architecture diagram.
Core implementation	Load balancer, master scheduler, workers, request/response models, and startup scripts.
Enhancement and fault tolerance	RAG module, Ollama integration, GPU metrics, and failure simulation endpoints.
Testing and finalization	Load, RAG, strategy, worker failover, master failover, README, report, and PDF documentation.

3. System Architecture

The architecture is split into independent services. This mirrors a production distributed system while remaining practical for a course project. Each service has a narrow responsibility and communicates over HTTP, making the design easy to run locally or across two physical MacBooks.



Generated Figure 1: DistributedLLM service architecture and default ports.

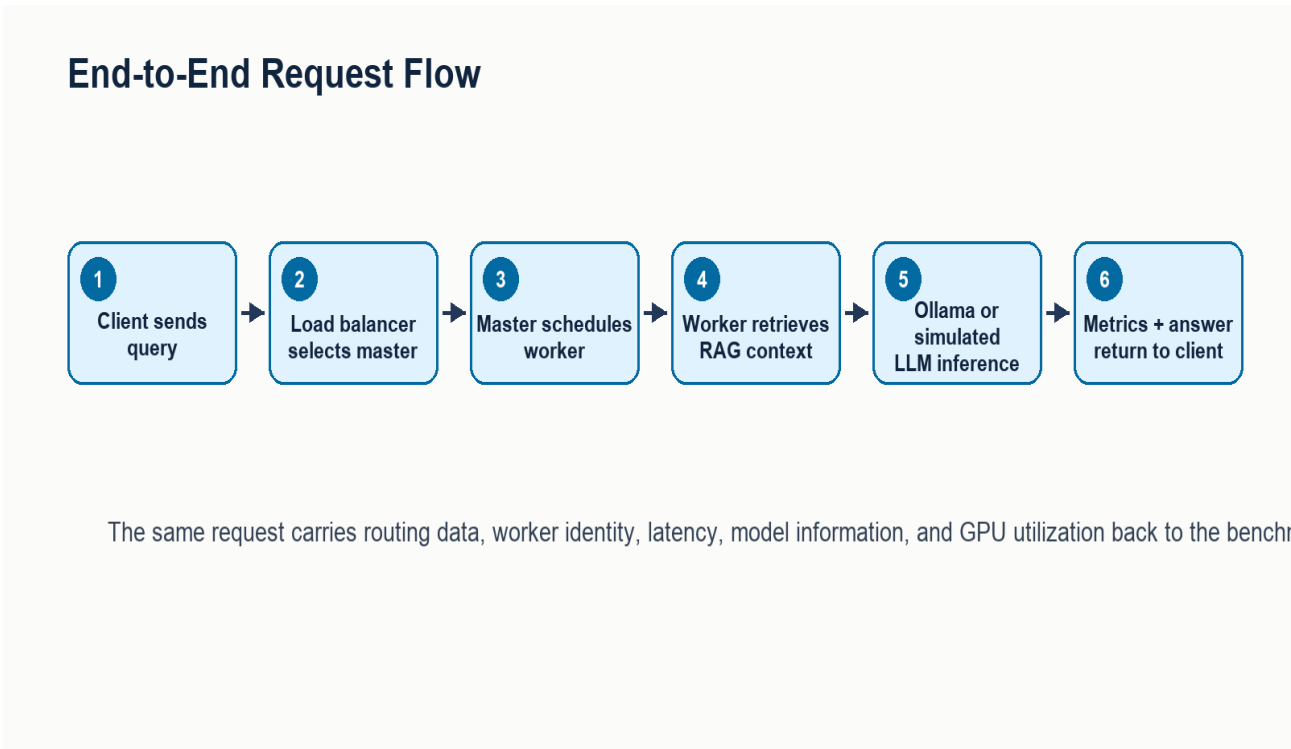
4. Component Design

Component	Files	Responsibility
Client	client/load_generator.py	Creates concurrent user requests and prints benchmark metrics.
Load Balancer	lb/load_balancer.py	Receives requests, monitors masters, selects a master, and retries failed masters.
Master Scheduler	master/scheduler.py	Tracks worker health, schedules tasks, retries failed workers, and adds master metrics.
GPU Worker	workers/gpu_worker.py	Processes requests, retrieves RAG context, runs inference, and reports worker metrics.

Component	Files	Responsibility
LLM Layer	llm/inference.py	Builds prompts and calls Ollama or simulation inference.
RAG Layer	rag/*.py	Ingests text/PDFs, chunks documents, stores vectors, and retrieves context.
Shared Models	common/*.py	Defines request/response models and GPU metric helpers.
Scripts	scripts/*.sh	Starts services and runs demo, load, RAG, and failover scenarios.

Request Flow

A request starts at the benchmark client and ends with an answer that includes routing and performance metadata. The system records which master and worker served the request, how long it took, which model answered, and what GPU metric source was used.



Generated Figure 2: End-to-end request lifecycle.

5. Load Balancing and GPU Task Distribution

The load balancer keeps a list of master nodes and refreshes their health through `/health``. The default Round Robin policy distributes requests evenly. Least Connections uses active tasks plus in-flight assignments so concurrent requests do not overload a recently selected master. Load-Aware routing adds GPU utilization and request history to the score, making the selection more sensitive to live resource usage.

- Round Robin is simple and fair when masters have similar capacity.
- Least Connections reacts to temporary load spikes.
- Load-Aware routing considers active load, GPU utilization, and request history.
- Masters then schedule work across their workers using round robin among alive workers.

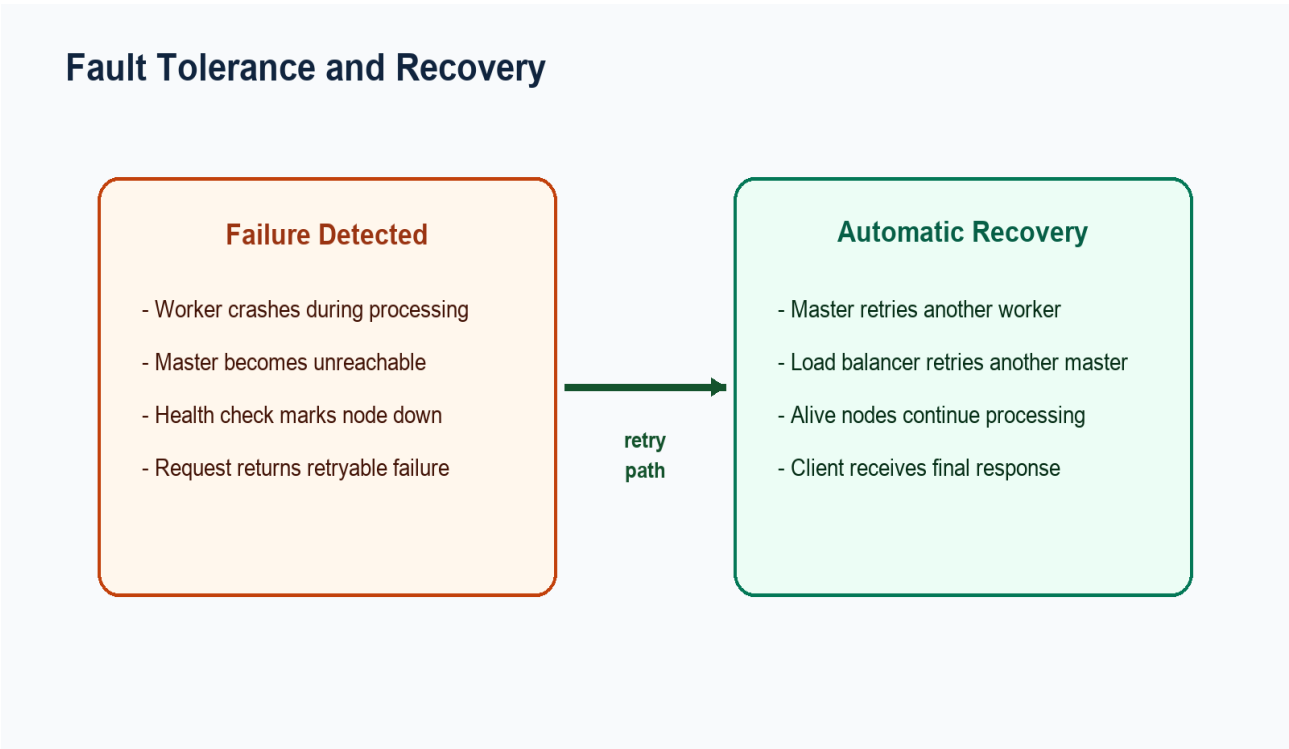
6. LLM Inference and RAG Integration

Workers use the RAG retriever before inference. If relevant document chunks are found, the prompt builder uses that context as the primary source. If no context is found or the RAG service is unavailable, the LLM still answers from general knowledge or simulation fallback. This satisfies the project requirement to enhance LLM responses with contextual information from a knowledge base.

- PDF ingestion uses `pdftotext` to extract text.
- Text is chunked with overlap to preserve context across boundaries.
- Qdrant stores chunk vectors and source metadata.
- Retrieval combines vector search with lexical keyword filtering.
- Ollama mode uses a local model; simulation mode is used for high-concurrency stress tests.

7. Fault Tolerance

Fault tolerance is implemented through health checks, failure marking, and retry paths. If a worker fails, the master retries another worker. If a master fails or cannot complete a request, the load balancer retries another master. This keeps the system available as long as at least one valid master-worker path remains alive.



Generated Figure 3: Failure detection and recovery paths.

8. Testing and Evaluation

Test Area	Example Script	Purpose
Load testing	<code>./scripts/test_load_simulation.sh 1000</code>	Simulate 1000 concurrent requests in fast mode.

Test Area	Example Script	Purpose
Ollama inference	<code>./scripts/test_ollama_10.sh</code>	Run real local LLM requests.
RAG retrieval	<code>./scripts/test_rag_retrieval.sh</code>	Check document retrieval quality.
Strategy comparison	<code>./scripts/test_strategies.sh 100</code>	Compare routing strategies.
Worker failover	<code>./scripts/test_worker_failover_simulation.sh</code>	Verify worker retry behavior.
Master failover	<code>./scripts/test_master_down_failover_ollama.sh</code>	Verify load balancer retry behavior.
Distributed mode	<code>./scripts/test_distributed_load.sh 10</code>	Run across two MacBooks.

Evaluation focuses on throughput, latency, success rate, master distribution, worker distribution, model distribution, and GPU utilization. Simulation mode is suitable for the 1000+ concurrency requirement, while Ollama mode demonstrates real inference at smaller scale because local model execution is much slower on laptop hardware.

Sample Output Fields

- Total users, successful requests, failed requests, and total runtime.
- Throughput in requests per second.
- Average, minimum, and maximum latency.
- Master and worker distribution.
- Model distribution in multi-machine mode.
- GPU utilization summary and metric source.

9. Limitations and Future Work

Current limitations:

- Workers represent GPU nodes, but the repository does not directly implement CUDA, NVIDIA, or Metal kernels.
- macOS GPU telemetry is device-level, not per-worker-process utilization.
- Simulation mode validates routing scalability but not real LLM throughput.
- Ollama mode depends on local hardware and model size.
- Local Qdrant storage is convenient for demos but not a production vector database deployment.
- Retries are request-level and do not provide durable queues or exactly-once processing.

Future work:

- Add durable queues for stronger crash recovery.
- Add a dashboard for live cluster metrics.
- Add automated tests with pytest.

- Add Docker Compose for reproducible deployment.
- Add authentication and rate limiting.
- Use neural embedding models for stronger semantic retrieval.
- Deploy Qdrant as a standalone distributed service.

10. Conclusion

DistributedLLM meets the core requirements of the CSE354 project description. It demonstrates a working distributed model for high-concurrency LLM requests with load balancing, master-worker scheduling, RAG integration, monitoring, and fault-tolerance behavior. The system is intentionally practical: it can run locally for stress testing, use Ollama for real LLM inference, and run across two MacBooks for a physical distributed demo.

References

- CSE354 Distributed Computing project specification, 2nd Semester 2025/2026.
- DistributedLLM GitHub repository: <https://github.com/karimmarwanw/DistributedLLM>
- FastAPI documentation: <https://fastapi.tiangolo.com/>
- Ollama documentation: <https://ollama.com/>
- Qdrant documentation: <https://qdrant.tech/documentation/>